파이썬에서의 파일 처리

파일 처리란 프로그래밍 인터페이스를 통해 파일을 생성, 열기, 읽기, 쓰기, 닫기 등의 작업을 수행하는 프로세스를 말합니다. 프로그램과 저장 장치의 파일 시스템 간의 데이터 흐름을 관리하여 데이터가 안전하고 효율적으로 처리되도록 하는 것을 포함합니다.

파일 처리가 필요한 이유는 무엇입니까?

- 프로그램이 종료된 후에도 데이터를 영구적으로 저장합니다.
- .txt, .csv, .json 등과 같은 외부 파일에 액세스하려면
- 많은 메모리를 사용하지 않고 대용량 파일을 효율적으로 처리합니다.
- 구성 읽기나 출력 저장과 같은 작업을 자동화합니다.
- 실제 애플리케이션과 도구에서 입출력을 처리합니다.

파이썬에서 파일 열기

파일을 열려면 인수로 file-path와 mode가 필요한 open() 함수를 사용하면 됩니다.

통사론:

```
file = open('filename.txt', 'mode')
```

- **filename.txt:** name (or path) of the file to be opened.
- **mode:** mode in which you want to open the file (read, write, append, etc.).

참고: *모드를 지정하지 않으면 Python은 기본적으로* ***'r' (읽기 모드)을 사용합니다.***

기본 예제: 파일 열기

```
f = open("geek.txt", "r")
print(f)
```

이 코드는 geeks.txt 파일을 읽기 모드로 엽니다. 파일이 있으면 성공적으로 연결하고, 없으면 오류를 발생시킵니다.

파일 닫기

파일 사용이 끝나면 파일을 닫는 것이 중요합니다. file.close() 메서드는 파일을 닫고 시스템 리소스를 해제하여 변경 사항이 제대로 저장되도록 합니다(쓰기의 경우).

```
file = open("geeks.txt", "r")
# Perform file operations
```

```
file.close()
```

파일 속성 확인

파일이 열리면 일부 속성을 확인할 수 있습니다.

```
f = open("geek.txt", "r")

print("Filename:", f.name)
print("Mode:", f.mode)
print("Is Closed?", f.closed)

f.close()
print("Is Closed?", f.closed)
```

산출:

설명:

- **f.name:** 열린 파일의 이름을 반환합니다(이 경우 "demo.txt").
- **f.mode:** 파일이 열린 모드를 알려줍니다. 여기서는 'r'이 읽기 모드를 의미합니다.
- **f.closed:** 부울 값을 반환합니다. 파일이 현재 열려 있으면 Fasle이고, 그렇지 않으면 True입니다.

파일 읽기

file.read() 를 사용하면 파일의 전체 내용을 읽어 파일을 읽을 수 있습니다 . 파일을 읽은 후에는 **file.close()를** 사용하여 파일을 닫을 수 있습니다 . 이 함수는 시스템 리소스를 확보하기 위해 파일을 읽은 후 닫습니다.

예: 읽기 모드에서 파일 읽기(r)

```
file = open("geeks.txt", "r")
content = file.read()
print(content)
file.close()
```

산출:

with 문 사용

파일을 수동으로 열고 닫는 대신, with 문을 사용하면 자동으로 닫힙니다. 이렇게 하면 파일 손상 및 리소스 누출 위험을 줄일 수 있습니다.

예: " Hello, World! "**라는 텍스트를 포함하는** geeks.txt 라는 파일이 있다고 가정해 보겠습니다 .

```
with open("geeks.txt", "r") as file:
    content = file.read()
    print(content)
```

산출:

파일을 닫을 때 예외 처리

파일 작업 중에 오류가 발생하더라도 파일이 제대로 닫히도록 예외를 처리하는 것이 중요합니다 .

```
try:
    file = open("geeks.txt", "r")
    content = file.read()
    print(content)
finally:
    file.close()
```

산출:

설명:

- **try:** 오류를 일으킬 수 있는 코드를 처리하는 블록을 시작합니다.
- **open():** 파일을 읽기 모드로 엽니다.
- **read():** 파일의 내용을 읽습니다.
- **finally:** 그 안에 있는 코드가 무슨 일이 있어도 실행되도록 보장합니다.
- **close():** 리소스를 해제하기 위해 파일을 안전하게 닫습니다.

Python의 파일 모드

Python에서 파일을 다룰 때, 파일 모드는 Python에게 파일에 어떤 종류의 작업(읽기, 쓰기 등)을 수행할지 알려줍니다. open() 함수의 두 번째 인수로 모드를 지정합니다.

Python의 다양한 파일 모드

다음은 Python의 다양한 파일 모드 유형과 설명입니다.

Mode	설명
‘r’	읽기 전용. 파일이 없으면 I/O 오류가 발생합니다.
'r+'	읽고 쓰기가 가능합니다. 파일이 없으면 I/O 오류가 발생합니다.
'w'	쓰기 전용입니다. 파일이 있으면 덮어쓰고, 없으면 새 파일을 만듭니다.
'w+'	읽고 쓰기. 파일을 덮어쓰거나 새 파일을 만듭니다.
‘a’	추가 전용입니다. 끝에 데이터를 추가합니다. 파일이 없으면 생성합니다.
'a+'	읽고 추가합니다. 포인터는 끝에 있습니다. 파일이 없으면 생성합니다.
‘rb’	이진 모드로 읽습니다. 파일이 존재해야 합니다.
'rb+'	이진 모드로 읽고 쓸 수 있습니다. 파일이 존재해야 합니다.
'wb'	이진수로 작성합니다. 덮어쓰거나 새로 만듭니다.
'wb+'	이진 형식으로 읽고 쓸 수 있습니다. 덮어쓰거나 새로 만듭니다.
‘ab’	이진 형식으로 추가합니다. 파일이 없으면 생성합니다.
'ab+'	이진 형식으로 읽고 추가합니다. 파일이 없으면 생성합니다.

공통 모드의 예

다음과 같은 내용을 가진 **example.txt** 라는 파일이 있다고 가정해 보겠습니다 .

1. 읽기 모드('r')

이 모드를 사용하면 파일을 읽기 전용으로 열 수 있습니다. 파일이 없으면 FileNotFoundError 가 발생합니다 .

예: 이 예에서 'example.txt'라는 파일을 읽기 모드('r')로 열고, 'with' 문을 사용하여 파일의 내용을 읽어 'content' 변수에 저장합니다. 이렇게 하면 사용 후 자동으로 파일을 닫아 적절한 리소스 관리를 보장합니다.

```
with open('example.txt', 'r') as file:
    content = file.read()
```

산출:

2. 쓰기 모드('w')

파일을 쓰기 전용으로 엽니다. 파일이 있으면 내용이 삭제되고, 없으면 새 파일이 생성됩니다.

예: 이 예에서는 'example.txt'라는 이름의 파일이 쓰기 모드('w')로 열리고, 문자열 'Hello, world!'가 파일에 기록됩니다.

```
with open('example.txt', 'w') as file:
    file.write('Hello, world!')
```

출력 (쓰기 후 파일 내용):

> ***참고: 이 코드를 실행한 후 "example.txt"*** *파일을 열면 이전 콘텐츠인 "Hello Geeks"가 삭제되고* ***"Hello, world!"*** *라는 텍스트가 포함되어 있는 것을 볼 수 있습니다 .*

3. 추가 모드('a')

기존 데이터를 삭제하지 않고 파일을 열어 마지막에 내용을 추가합니다. 파일이 없으면 새 파일을 만듭니다.

예: 이 예에서 'example.txt'라는 이름의 파일이 추가 모드('a')로 열리고, 문자열 '\n This is a new line.'이 파일 끝에 기록됩니다.

```
with open('example.txt', 'a') as file:
    file.write('\nThis is a new line.')
```

산출:

4. 바이너리 모드('b')

이미지나 오디오와 같은 텍스트가 아닌 파일에 사용됩니다. 항상 'r', 'w' 또는 'a'와 함께 사용됩니다.

예: 이 예제에서는 'image.png'라는 파일을 이진 읽기 모드('rb')로 엽니다. 'read()' 메서드를 사용하여 파일에서 이진 데이터를 읽어 'data' 변수에 저장합니다.

```
with open('image.png', 'rb') as file:
    data = file.read()
    # Process the binary data
```

5. 읽기 및 쓰기 모드('r+')

파일을 읽고 쓸 수 있도록 엽니다. 파일의 처음부터 시작합니다. 파일이 존재하지 않으면 **FileNotFoundError를 발생시킵니다.**

```
with open('example.txt', 'r+') as file:
    content = file.read()
    file.write('\nThis is a new line.')
```

출력:

6. 쓰기 및 읽기 모드('w+')

이 모드를 사용하면 파일을 읽고 쓸 수 있습니다. 파일이 이미 존재하면 파일의 길이를 0으로 자릅니다. 파일이 존재하지 않으면 새 파일을 생성합니다.

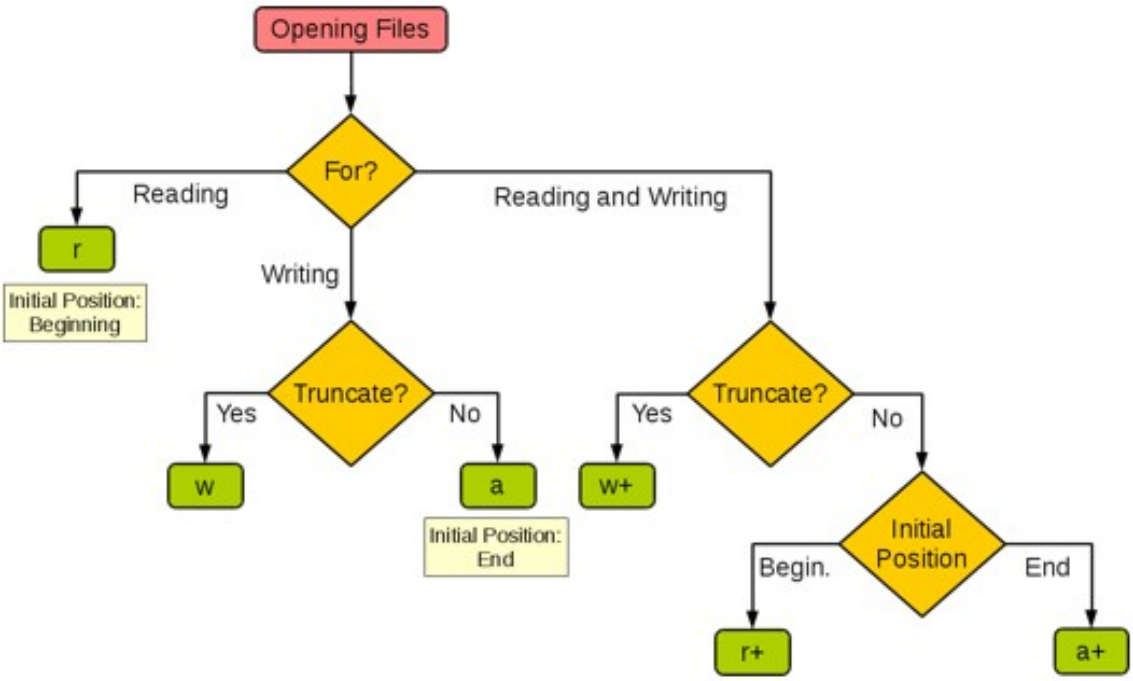
예: 이 예에서 'example.txt'라는 파일은 쓰기 및 읽기 모드('w+')로 열립니다.

```
with open('example.txt', 'w+') as file:
    file.write('Hello, world!')
    file.seek(0)
    content = file.read()
```

산출:

설명: 이 코드의 출력은 "Hello, world!"입니다. 파일이 잘리고 읽기 전에 포인터가 처음으로 이동했기 때문에 파일의 내용은 기록된 내용 그대로입니다. 따라서 내용에는 "Hello, world!"라는 문자열이 포함됩니다.

파이썬 파일모드 선택



파이썬에서 파일 읽기

Python 에서 파일을 읽는다는 것은 텍스트, 바이너리 데이터 또는 CSV나 JSON과 같은 특정 데이터 형식 등 파일의 내용에 접근하고 가져오는 것을 의미합니다. Python은 파일을 효율적으로 읽을 수 있는 내장 함수와 메서드를 제공합니다.

예제 파일: geeks.txt

```
Hello World
Hello GeeksforGeeks
```

Python에서 기본 파일 읽기

기본적인 파일 읽기에는 파일을 열고, 파일의 내용을 읽고, 시스템 리소스를 확보하기 위해 파일을 제대로 닫는 작업이 포함됩니다.

단계:

- **파일을 엽니다.** open("filename", "mode")는 지정된 모드(예: 읽기 모드 "r")로 파일을 엽니다.
- **콘텐츠 읽기:** read(), readline() 또는 readlines() 메서드를 사용합니다.
- **파일을 닫습니다.** close()는 시스템 리소스가 해제되도록 합니다.

예: 전체 파일 읽기

```
# Open the file in read mode
file = open("geeks.txt", "r")

# Read the entire content of the file
content = file.read()

print(content)

# Close the file
file.close()
```

산출:

설명: 이 코드는 **geeks.txt**를 읽기 모드로 열고, 모든 내용을 문자열로 읽어서 인쇄한 다음, 리소스를 해제하기 위해 파일을 닫습니다.

모범 사례: with 문 사용

with open(...)을 사용하면 파일이 자동으로 닫힙니다.

```
with open("geeks.txt", "r") as file:
    content = file.read()
    print(content)
```

설명: 이 코드는 블록이 종료되면 파일이 자동으로 닫히도록 하여 리소스 누수를 방지합니다.

파일을 줄별로 읽기

특히 전체 내용을 한 번에 읽기 어려운 큰 파일의 경우, 파일을 한 줄씩 읽어야 할 수 있습니다. 다음 두 가지 방법을 사용하면 됩니다.

- **파일의** 각 줄을 반복합니다.
- **line.strip():** 줄 바꿈 문자를 포함하여 앞뒤의 공백을 제거합니다.

예제 1: 루프를 사용하여 줄별로 읽기

```
# Open the file in read mode
file = open("geeks.txt", "r")

# Read each line one by one
for line in file:
    print(line.strip())  # .strip() to remove newline characters

# Close the file
file.close()
```

산출:

설명: 이 방법은 파일의 각 줄을 한 번에 하나씩 읽고 앞뒤 공백을 제거한 후 인쇄합니다.

예제 2: readline() 사용

file.readline() 은 한 번에 한 줄씩 읽습니다. **while line은** 더 이상 읽을 줄이 없을 때까지 계속 읽습니다.

```
# Open the file in read mode
file = open("geeks.txt", "r")
```

```
# Read the first line
line = file.readline()

while line:
    print(line.strip())
    line = file.readline()  # Read the next line

# Close the file
file.close()
```

산출:

설명: 이 메서드는 readline()을 사용하여 한 번에 한 줄씩 읽습니다. 이는 파일을 청크로 처리할 때 유용합니다.

Python에서 바이너리 파일 읽기

이진 파일은 텍스트로 읽을 수 없는 형식으로 데이터를 저장합니다. 여기에는 이미지, 실행 파일 또는 텍스트가 아닌 모든 데이터가 포함될 수 있습니다. 이진 파일을 읽기 위해 다음과 같은 방법을 사용합니다 .

- **open("example.bin", "rb"):** example.bin 파일을 읽기 바이너리 모드로 엽니다.
- **file.read():** 파일의 전체 내용을 바이트로 읽습니다.
- **file.close():** 시스템 리소스를 확보하기 위해 파일을 닫습니다.

예: 바이너리 파일 읽기

```
# Open the binary file in read binary mode
file = open("geeks.txt", "rb")

# Read the entire content of the file
content = file.read()

# Print the content (this will be in bytes)
print(content)

# Close the file
file.close()
```

산출:

```
안녕하세요, GeeksforGeeks입니다.
```

설명: 이 코드는 바이너리 모드("rb")로 파일을 읽고 내용을 바이트로 출력합니다. 이는 텍스트가 아닌 파일을 처리하는 데 필요합니다.

파일의 특정 부분 읽기

때로는 파일의 특정 부분, 예를 들어 처음 몇 바이트, 특정 줄 또는 여러 줄의 범위만 읽어야 할 수도 있습니다. **예: 처음 N 바이트 읽기**

```
# Open the file in read mode
```

```
file = open("geeks.txt", "r")

# Read the first 10 bytes
content = file.read(10)

print(content)

# Close the file
file.close()
```

산출:

```
안녕하세요  세상
```

설명: 이 코드는 파일의 처음 10자만 읽으며, 파일 내용을 미리 보는 데 유용합니다.

Python에서 파일에 쓰기

Python에서 파일에 쓴다는 것은 프로그램에서 생성된 데이터를 시스템의 파일에 저장하는 것을 의미합니다. 이 글에서는 Python에서 파일에 쓰는 방법을 자세히 다룹니다.

파일 생성

파일을 생성하는 것은 파일에 데이터를 쓰기 전 첫 번째 단계입니다. Python에서는 다음 세 가지 모드를 사용하여 파일을 생성할 수 있습니다.

- **쓰기("w") 모드:** 이 모드는 파일이 존재하지 않으면 새 파일을 생성합니다. 파일이 이미 존재하면 기존 내용을 삭제하고 처음부터 다시 시작합니다.
- **추가("a") 모드:** 이 모드는 파일이 존재하지 않으면 새 파일을 생성합니다. 파일이 존재하면 기존 데이터를 수정하지 않고 파일 끝에 새 내용을 추가합니다.
- **배타적 생성("x") 모드:** 이 모드는 파일이 존재하지 않는 경우에만 새 파일을 생성합니다. 파일이 이미 존재하는 경우 FileExistsError를 발생시킵니다.

예:

```
# Write mode: Creates a new file or truncates an existing file

with open("file.txt", "w") as f:
    f.write("Created using write mode.")

f = open("file.txt","r")
print(f.read())

# Append mode: Creates a new file or appends to an existing file

with open("file.txt", "a") as f:
    f.write("Content appended to the file.")

f = open("file.txt","r")
print(f.read())

# Exclusive creation mode: Creates a new file, raises error if file exists

try:
    with open("file.txt", "x") as f:
        f.write("Created using exclusive mode.")
except FileExistsError:
    print("Already exists.")
```

산출

기존 파일에 쓰기

이미 존재하는 파일을 수정하거나 새로운 내용을 추가하려면 다음 두 가지 방법을 사용할 수 있습니다.

쓰기 모드("w"): 기존 콘텐츠를 덮어씁니다.

writelines() : 한 번의 호출로 문자열 목록을 파일에 쓸 수 있습니다.

예:

```
# Writing to an existing file (content will be overwritten)
with open("file1.txt", "w") as f:
    f.write("Written to the file.")

f = open("file1.txt","r")
print(f.read())

# Writing multiple lines to an existing file using writelines()
s = ["First line of text.\n", "Second line of text.\n", "Third line of text.\n"]

with open("file1.txt", "w") as f:
    f.writelines(s)

f = open("file1.txt","r")
print(f.read())
```

산출

```
파일에 기록됨.
첫 번째 텍스트 줄.
두 번째 줄의 텍스트.
세 번째 줄의 텍스트.
```

설명:

- **open("example.txt", "w"):** example.txt 파일을 쓰기 모드로 엽니다. 파일이 존재하는 경우, 기존 내용은 삭제되고 새 데이터로 대체됩니다.
- **file.write("파일에 기록되었습니다."):** 새 내용을 파일에 기록합니다.
- **file.writelines(lines):** 이 메서드는 문자열 목록을 받아서 파일에 씁니다. 단일 문자열을 쓰는 write()와 달리, writelines()는 목록의 각 요소를 차례로 씁니다. 줄 사이에 줄 바꿈 문자를 자동으로 추가하지 않으므로, 적절한 줄 바꿈을 위해 문자열에 \n을 포함해야 합니다.

바이너리 파일에 쓰기

텍스트가 아닌 데이터(예: 이미지, 오디오 또는 기타 이진 데이터)를 다룰 때 Python에서는 이진 모드로 파일에 쓸 수 있습니다. 이진 데이터는 텍스트로 인코딩되지 않으며, 이진 쓰기 모드(`"wb"`)를 사용하면 파일 내용이 원시 바이트로 처리됩니다.

예:

```python
# Writing binary data to a file
bin = b'\x00\x01\x02\x03\x04'

with open("file.bin", "wb") as f:
    f.write(bin)

f = open("file.bin","r")
print(f.read())
```

산출

```
┌┘
```

설명:

- **bin= b'\x00\x01\x02\x03\x04':** 문자열 앞의 b는 이진 데이터임을 나타냅니다. 각 쌍은 바이트 값을 나타냅니다.
- **open("file.bin", "wb"):** file.bin 파일을 이진 쓰기 모드로 엽니다. 파일이 없으면 Python이 파일을 생성합니다.
- **file.write(bin):** 바이너리 데이터를 원시 바이트로 파일에 씁니다.